Ouvidoria Inteligente

Relatório técnico de representações vetoriais e triagem semântica — resultados experimentais

O protótipo recupera manifestações por significado, identifica candidatas a duplicatas e divide textos longos em trechos analisáveis. A entrega contém três notebooks executados de ponta a ponta nesta versão, aplicação Streamlit, núcleo compartilhado e arquivos de dados. Os resultados abaixo foram medidos sobre a base demonstrativa do projeto, com os dois modelos reais indicados.

Dados e proveniência

O arquivo manifestacoes.json foi criado como base **sintética**, com autorização do solicitante, porque a base original não constava dos materiais disponíveis. Ele segue exatamente os campos id, data, categoria_oficial e texto. Contém 40 registros M001 a M040, cinco categorias e textos entre 50 e 800 caracteres. M010, M016, M024, M033 e M040 têm mais de 500 caracteres. Nenhum registro representa uma pessoa real.

O gabarito exaustivo contém M003-M017, M008-M022 e M005-M026: três pares disjuntos abrangem seis registros, isto é, 15% da base. Se contadas apenas as cópias adicionais, são três registros, ou 7,5%; a convenção adotada está explícita em PROVENIENCIA.json. Pares da mesma categoria, mas de eventos ou locais diferentes, foram considerados negativos.

Pipeline e modelos

BoW e TF-IDF compartilham normalização de caixa e acentos, sem remoção de stopwords nesta comparação. Os embeddings são produzidos por MiniLM multilíngue e DistilUSE multilíngue, ambos baixados e executados localmente na CPU. A categoria oficial não entra no modelo: serve apenas para inspeção da projeção. Textos que excedem a capacidade do modelo são divididos em janelas de tokens e agregados por média normalizada, evitando perda silenciosa por truncamento.

O aplicativo usa cache de recursos para os modelos e cache de dados para embeddings. Busca e matriz usam produto escalar de vetores L2 normalizados, equivalente ao cosseno. A interface permite trocar o modelo e o top-k. A matriz completa, PCA, t-SNE e o laboratório de chunking estão em abas próprias.

Autoria individual

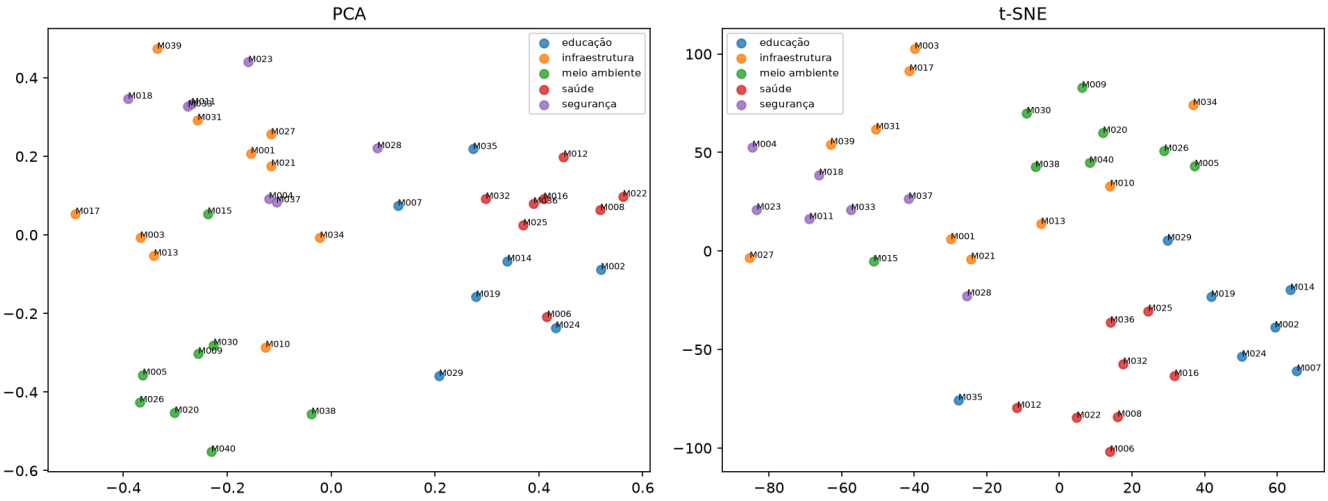
Fernando Amorim Pontes Neto é responsável pelo desenvolvimento, organização da base demonstrativa, execução dos experimentos e relatório. Os exemplos didáticos fornecidos orientaram a estrutura, sem transformar as tarefas de e-commerce em entregas adicionais.

Comparação dos pares obrigatórios

Par	BoW	TF-IDF	MiniLM	DistilUSE
M003 / M017	0.401	0.359	0.794	0.834
M008 / M022	0.500	0.492	0.921	0.856
M008 / M031	0.136	0.034	0.198	0.131

Os dois primeiros pares descrevem a mesma ocorrência com vocabulário diferente (buraco na Av. Brasil; falta de médico no PSF São Lucas); M008-M031 contrapõe atendimento de saúde e iluminação de praça, sem relação temática. BoW e TF-IDF já separam bem esse contraste por sobreposição lexical, mas dependem de palavras compartilhadas. Os dois modelos de embeddings recuperam a relação de significado nos pares positivos com folga (0,79 a 0,92) e mantêm o par negativo bem abaixo (0,13 a 0,20), sem inverter nenhuma ordem entre os três pares.

Espaço vetorial e categorias



No espaço original (MiniLM), o cosseno médio entre registros da mesma categoria foi 0.452; entre categorias, 0.281. A média intra-categoria é maior, o que sugere algum alinhamento temático, mas a diferença é modesta e não prova clusters separados — a inspeção visual de PCA e t-SNE confirma pontos da mesma categoria espalhados pelo plano, e não agrupados. PCA projeta a variância em duas dimensões; t-SNE evidencia vizinhanças locais, mas distâncias globais e tamanhos dos grupos não devem ser lidos literalmente.

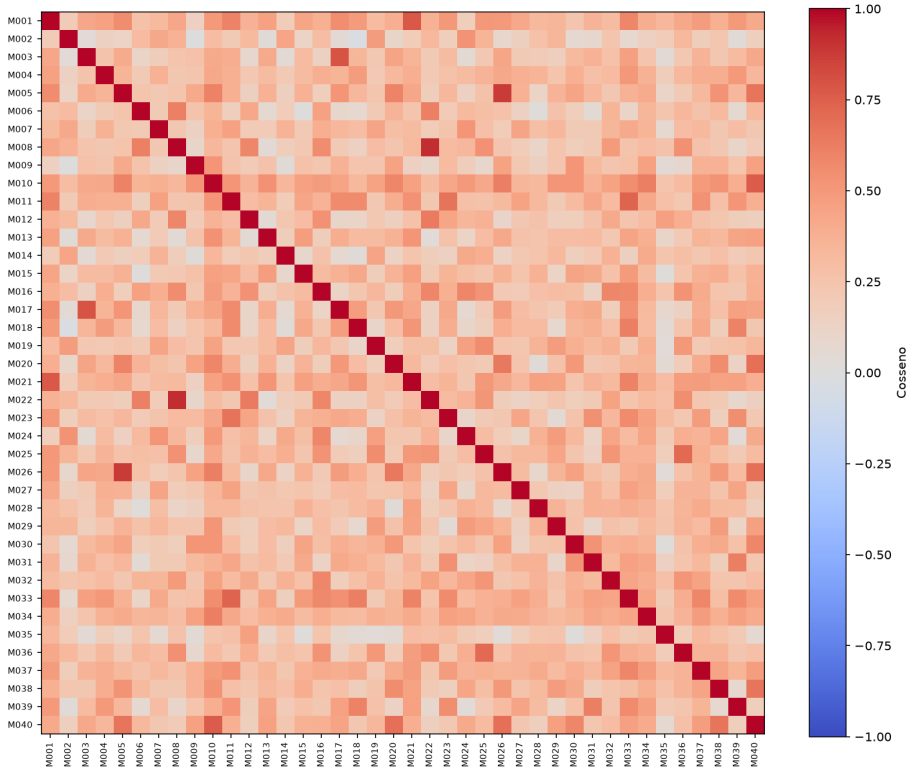
Detecção de duplicatas

Com similaridade estritamente maior que 0,85 (MiniLM), o detector encontrou 2 pares: 2 verdadeiros positivos, 0 falsos positivos e 1 falsos negativos. Precisão = 1.000; recall = 0.667; F1 = 0.800. Os cálculos excluem a diagonal e contam cada par uma única vez.

Verdadeiros positivos: M005-M026, M008-M022.
Falsos positivos: nenhum.
Falsos negativos: M003-M017.

O par perdido, M003-M017, descreve o mesmo buraco na Av. Brasil com vocabulário quase sem sobreposição lexical direta ("buraco" / "esburacado", "desviarem" / "invadir a pista"); seu cosseno (0,794) ficou logo abaixo do limiar de 0,85. Isso mostra que 0,85 é conservador o suficiente para gerar um falso negativo real nesta base, mesmo sem produzir nenhum falso positivo — um comportamento de alta precisão às custas de recall, coerente com um limiar pensado para poupar revisão humana de pares improváveis.

Matriz completa e decisão de limiar



A diagonal vale 1 e é excluída da seleção. O percentil 90 dos 780 pares únicos foi 0.502 — bem abaixo de 0,85. Esse percentil seleciona por construção cerca de 10% dos pares (78 candidatos, todos os 3 verdadeiros incluídos, mas com 75 falsos positivos), não a proporção real de duplicatas: usá-lo como limiar trocaria os 0 falsos positivos obtidos com 0,85 por dezenas de pares para revisão manual desnecessária.

Limiar	Pares	Precisão	Recall	F1
0.65	12	0.250	1.000	0.400
0.70	7	0.429	1.000	0.600
0.75	5	0.600	1.000	0.750
0.80	2	1.000	0.667	0.800
0.85	2	1.000	0.667	0.800
0.90	1	1.000	0.333	0.500
0.95	0	0.000	0.000	0.000

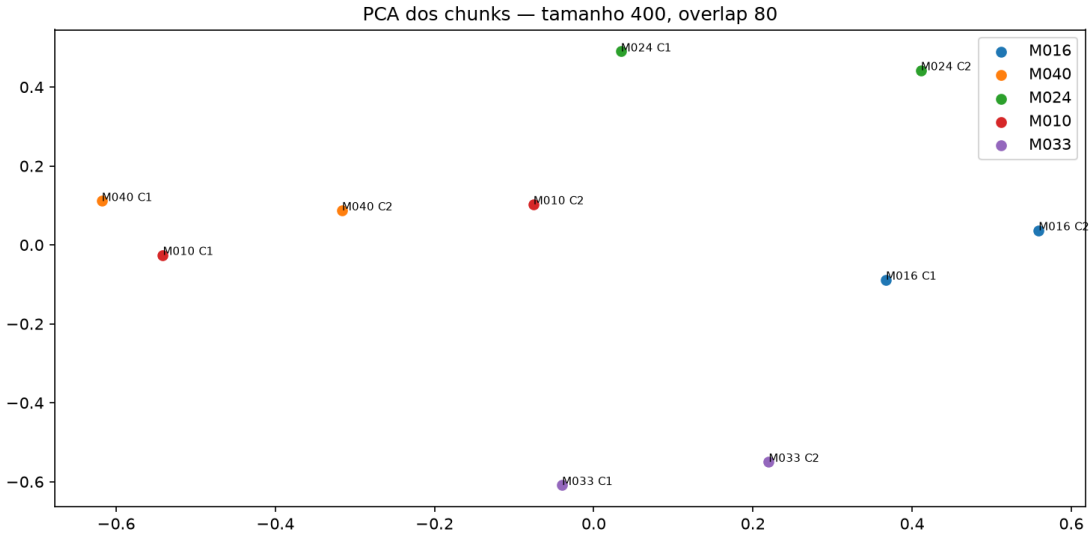
A tabela de sensibilidade mostra a troca clássica entre precisão e recall: até 0,75, os três pares do gabarito são todos recuperados (recall = 1,0), mas ao custo de falsos positivos crescentes conforme o limiar cai (5 pares detectados em 0,75, contra 2 em 0,80/0,85). Entre 0,76 e 0,89, M003-M017 (cosseno 0,794) já não passa e o recall cai para 0,667; acima de 0,90, o detector fica ainda mais conservador e passa a perder também M005-M026 (cosseno 0,882). O F1 máximo da faixa testada (0,80) ocorre empatado em 0,80 e 0,85. Nenhum limiar único desta base atinge precisão e recall perfeitos ao mesmo tempo; a escolha de 0,85 prioriza não incomodar o revisor humano com pares improváveis, ao custo de exigir um segundo mecanismo (ex.: revisão de pares logo abaixo do limiar) para não perder casos como M003-M017.

Chunking das cinco manifestações longas

RecursiveCharacterTextSplitter tenta separadores de parágrafo, linha, frase e espaço, com corte por caractere como último recurso. Comparamos (240, 0), (240, 60) e (400, 80) sobre M010, M016, M024, M033 e M040.

Tamanho / overlap	Chunks	Cosseno vizinhos	Overlap real médio
240 / 0	22	0.449	0.0
240 / 60	22	0.449	0.0
400 / 80	10	0.607	0.0

Resultado inesperado e relevante: ao manter chunk_size=240 e subir o overlap pedido de 0 para 60 caracteres, o número de chunks (22), o cosseno médio entre vizinhos (0.449) e o overlap real medido (0.0 caracteres) não mudaram. Nestes textos, os separadores de frase (". ") já produzem cortes que cabem em 240 caracteres sem precisar repetir conteúdo entre trechos vizinhos; o parâmetro de overlap do splitter só entra em ação quando teria de mesclar unidades adjacentes, o que não ocorreu aqui. Ou seja, o overlap solicitado foi silenciosamente ignorado nesta configuração — um limite prático do splitter recursivo baseado em caracteres, não um resultado de melhor compreensão semântica.



Em (400, 80), que de fato aplicou overlap (10 chunks, contra 22 nas configurações de 240), chunks da mesma manifestação tiveram cosseno médio 0.607, contra 0.376 entre manifestações diferentes — uma separação clara, também visível na projeção PCA, onde os dois chunks de cada manifestação ficam próximos entre si e afastados dos demais.

Para textos deste tamanho e estilo (frases curtas, poucos parágrafos), (400, 80) é a configuração que efetivamente varia o overlap e reduz a quantidade de trechos pela metade, mantendo local, problema e consequência mais frequentemente juntos no mesmo chunk. (240, 0/60) gera mais unidades menores, úteis para buscas de detalhe pontual, mas sem o benefício de overlap real nesta base — um ajuste de separadores ou o uso de overlap por token, e não por caractere, seria necessário para obter sobreposição efetiva em textos curtos como estes.

Reprodução e aprendizados

Instale requirements.txt, execute os três notebooks nesta pasta (nesta ordem: análise_comparativa, deteccao_duplicatas, chunking_manifestacoes) e inicie **streamlit run app_ouvidoria.py**. Os modelos são baixados no primeiro uso. Os testes cobrem pares únicos, limiar estrito, métricas, limites e cobertura dos chunks, projeção com um ponto e validação do JSON (python testes.py, 5/5 aprovados). Os principais

aprendizados desta execução: separar afinidade temática de duplicidade do evento; um limiar único de similaridade não é suficiente para cobrir toda a faixa de paráfrases sem custar falsos positivos; e a sobreposição de um splitter por caracteres depende dos separadores do texto, podendo não ter efeito algum em textos curtos — vale sempre medir o overlap real obtido, não apenas o solicitado.

Referências: Desafio_Ouvidoria_Inteligente.docx e materiais de apoio; sbert.net; docs.langchain.com/oss/python/integrations/splitters/recursive_text_splitter; scikit-learn.org/stable/modules/manifold.html.